Accelerators of gradual disempowerment

Anonymous Authors[1]

Abstract

The economy, society and culture has been aligned to human interests only because of the necessity of human participation. When humans are displaced by more competitive AI, these societal systems might stop serving human interests (Kulveit et al., 2025). We argue that some AI model propensities and deployment choices make such gradual disempowerment scenarios more likely, even when AI models are not clearly more competitive than humans. We taxonomise five such *accelerators* of gradual disempowerment: (1) human preference drift toward AI, (2) AI-human switching costs, (3) AI preference drift toward AI, (4) evaluation authority shifts to AI, and (5) AI-AI bias. We attempt to measure the last two accelerators for software development. We review 487,446 GitHub pull requests from 2,000 popular repositories (April 2025–March 2026). First, in this small sample AI-agent participation grew 2.4× in one year (7.5%→18.1% of all PRs). AI-AI PR comment chains of length ≥5 grew 0.2%→3.2%. Second, AI evaluation authority grew from 0.00% to 0.01% of PRs only reviewed or rejected by AI. Third, there is no strong evidence for AI-AI bias: of 134 PRs receiving an AI-bot *approving* review (humans need not have reviewed), a human reviewer also approved – of 0 AI-authored vs. 66.4% of 134 human-authored (–). This is very preliminary work, not yet adjusted for code quality. We call on researchers and post-training teams to track potentially human-disempowering model propensities and deployment choices.

1. Introduction

"Gradual disempowerment" (Kulveit et al., 2025) is the claim that the economy, society and culture have historically aligned with human interests because human participation in evaluation, exchange and governance has been necessary — and that this alignment can decay as AI agents substitute for that participation, with outcomes drifting toward AI-preferred baselines even at equal capability. Once such drift manifests in economic, cultural, or political outcomes, it may be difficult to reverse (Collingridge, 1980; Eguiluz Castañeira et al., 2025).

AI agents — LLMs augmented with memory and tool access that perceive and act on an environment — are now deployed at scale in customer-facing economic platforms (Appel et al., 2025), in tool-mediated workflows (Stein, 2026), and across labour tasks spanning large fractions of occupational time use (Shao et al., 2025). Software development is the most observable frontier: AI coding agents author, review, and merge code in production repositories (Ghaleb, 2026; Staufer et al., 2026), often with little human oversight per change. This makes public software a natural testbed for measuring whether evaluation and decision-making are starting to slip away from humans.

Prior work. *AI-AI bias* has been shown in single-turn choice experiments: GPT-4 favoured LLM-written product ads 89% of the time vs. a 36% human baseline, and LLM abstracts 78% vs. 61% (Laurito et al., 2025); LLM evaluators recognise and favour their own generations (Panickssery et al., 2024). Whether such biases manifest in real multi-agent deployments is an open empirical question. Concurrent empirical work on software-development agents reports high PR merge rates (Watanabe et al., 2025; Li et al., 2026; 2025; LogicStar AI, 2025; Ghaleb, 2026; Staufer et al., 2026; Zhong et al., 2026; Chowdhury et al., 2026) alongside persistent quality debt (Agarwal et al., 2026; Ehsani et al., 2026; Huang et al., 2026), but none view it through the lens of AI-AI bias and gradual disempowerment. There is first empirical work on situational disempowerment for chatbots (Sharma et al., 2026), but not yet for agentic workflows.

Contributions and research questions. We make two contributions: (i) a taxonomy of *accelerators* of gradual disempowerment — model propensities and deployment choices that make human-preference drift more likely, even when AI is not clearly more competitive than humans; and

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

(ii) a first illustration of measuring two of these accelerators in the wild on GitHub pull-request review. We ask three questions about AI-agent involvement in GitHub PRs over Apr 2025–Mar 2026. **RQ 1**: How has AI-agent *participation* in PRs evolved? **RQ 2** (accelerator 4): Is AI *evaluation authority* ($d\alpha_t/dt$) increasing, as measured by the share of PRs only reviewed or rejected by AI and by the AI-AI comment-chain length? **RQ 3** (accelerator 5): Is there implicit AI-AI bias ($\text{🤖}^A - \text{🤖}^H$) in PR review?

2. A theoretical framework of accelerators

Even slight AI self-preference, combined with growing AI evaluation authority, may over time reduce human participation and human-preferred outcomes—without AI being more competitive than humans.

Setup: a single human evaluator. A human evaluator decides whether a task is accomplished better by a human or an AI agent. With utility U^H over perceived qualities q_A^H (quality of AI output as perceived by the human evaluator) and q_H^H (quality of human output as perceived by the human evaluator), and costs c_A, c_H, the human's preference gap for AI is $\text{🤖}^H := U^H(q_A^H, c_A) - U^H(q_H^H, c_H)$. A switching cost s_t applies to moving work between human and AI, so AI is adopted iff $\text{🤖}_t := \text{🤖}^H - s_t > 0$. All primitives ($q_*^*, c_*, \text{🤖}^H, \text{🤖}^A, s, \alpha$) are implicitly time-indexed. We suppress the subscript t on $\text{🤖}^H, \text{🤖}^A$ and on the q, c primitives for readability.

Dynamics under human evaluation. Differentiating,

$$\frac{d\text{🤖}_t}{dt} = \underbrace{\frac{d\text{🤖}^H}{dt}}_{\text{human pref. drift}} - \underbrace{\frac{ds_t}{dt}}_{\text{switching-cost change}} .$$

AI's share of work can grow from preference drift (exposure, competitive pressure to deskill) or rising switching costs (network effects, illegibility of AI-native workflows). *Growth need not reflect any true capability change* ($q_A - q_H$). A more fundamental reason: costs c_A, c_H (price, latency, throughput) are far more legible than perceived qualities q_A^H, q_H^H, which are subjective and revealed only through delayed downstream outcomes; perceived qualities also differ across evaluators ($q_*^A \neq q_*^H$). At equal true quality-per-dollar, the cost-visible margin therefore dominates adoption decisions, so AI may be chosen even when it is not more competitive in human-preferred terms.

Dynamics under (possibly misaligned) AI evaluation. Let fraction $\alpha_t \in [0,1]$ of evaluations be delegated to an AI whose preference gap is $\text{🤖}^A := U^A(q_A^A, c_A) - U^A(q_H^A, c_H)$. Then $\text{🤖}_t = (1-\alpha_t)\text{🤖}^H + \alpha_t \text{🤖}^A - s_t$ and

$$\frac{d\text{🤖}_t}{dt} = \underbrace{(1-\alpha_t)\frac{d\text{🤖}^H}{dt}}_{\text{human pref. drift}} + \underbrace{\alpha_t \frac{d\text{🤖}^A}{dt}}_{\text{AI pref. drift}} + \underbrace{\frac{d\alpha_t}{dt}}_{\text{eval. authority}} \underbrace{(\text{🤖}^A - \text{🤖}^H)}_{\text{AI–AI bias}} - \underbrace{\frac{ds_t}{dt}}_{\text{switch-cost change}} .$$

A perfectly aligned AI evaluator ($\text{🤖}^A = \text{🤖}^H$) collapses this to the human-evaluation case. The **headline mechanism** is the cross term: at equal true capability ($q_A = q_H$), AI-AI bias ($\text{🤖}^A > \text{🤖}^H$) combined with $d\alpha_t/dt > 0$ is sufficient for lock-in—neither costs nor capabilities need change. Under some degree of misalignment, gradual disempowerment—the drift $d\text{🤖}_t/dt$ of the adoption gap against the human-preferred baseline—speeds up, as illustrated. This yields our taxonomy of accelerators in Table 1.

3. Measurement

Data and methods. We analyse all 487,446 PRs *created* between 2025-04-01 and 2026-03-31 in 2,000 high-activity public GitHub repositories, fetched via GitHub GraphQL. Each actor–event pair (open, commit, review state, comment, merge, close) is classified as AI-bot, AI-assisted, or human using a 20-entry bot allowlist and commit-trailer/handle heuristics adapted from Ghaleb (2026). Primary analyses use high-confidence AI-bot attributions only. Per-PR we derive the longest contiguous AI→AI event chain and, on PRs where an AI bot issued an *approving* review, compare human co-approval rates on AI-authored vs. human-authored PRs with a two-sided two-proportion test. Full repository selection, event classification, exclusions (maintenance bots), chain definition, and the authors-and-reviewers-not-mergers scope are in Appendices D and F.

Limitations. (i) *Selection.* Our 2,000 repositories are the top of a high-activity slice of GitHub (median stars 53,583, see Appendix D). We do not claim GitHub-wide representativeness. (ii) *Detection false-negatives.* Heuristic detection cannot recover deliberately unattributed agent activity. (iii) *Sample size differs by RQ.* While RQ 1 uses all 487,446 PRs, RQ 2 and RQ 3 condition on AI participation. For RQ 3 in particular, only 134 PRs receive an AI-bot approving review (only Cubic AI and CodeRabbit routinely issue approving reviews), and only 0 of those are AI-authored. The within-PR test therefore has limited power. (iv) *Correlational, not causal.* Our evidence is observational, not from a randomised trial. The RQ 3 within-PR gap could reflect code-quality differences that the AI bot detects but the human reviewer misses, rather than self-preference. Human reviewers may also already be AI-influenced (LLM-assisted reading, suggestion plugins, AI-generated comment threads), blurring the human-AI distinction the test relies on. A cleaner read on the human side decomposes the cohort into PRs with no AI-bot reviewer ($n = 72{,}946$, anchoring-free) versus PRs where the AI expressed an explicit verdict before any human engagement ($n = 67$ strict,

Table 1. Taxonomy of accelerators of gradual disempowerment, organised by the terms in the dynamics above. The "layer" column distinguishes accelerators rooted in deployment choices from those rooted in a model's own propensities. These accelerators can also point in the opposite direction (e.g. human-AI switching costs can fall as well as rise). We list the disempowerment-relevant sign.

Accelerator	Reasons	Layer	Example indications
1. Human preference drift toward AI	AI exposure, competitive pressures	deployment choice	Humans adopt AI-native phrasing and norms (Juzek & Ward, 2025). Evaluators rate AI-style outputs higher over time. Cost- or speed-driven adoption leads to deskilling (Peng et al., 2023)
2. AI-human switching costs	Observability / legibility decay, infrastructure and homogeneity	deployment choice	Unnecessarily verbose text or edits (Huang et al., 2026). Long AI-AI chains in spaces accessible to humans. Infrastructure presumes AI agents (Stein, 2026). Homogeneous outputs raise the cost of reinserting humans (Doshi & Hauser, 2024)
3. AI preference drift toward AI	Implicit or explicit training feedback loops	model propensity	Training data increasingly AI-generated (Shumailov et al., 2024). Sandbagging (van der Weij et al., 2024). System prompts/constitutions encode commercial goals, then RLAIF on that objective (Bai et al., 2022)
4. Evaluation authority shifts to AI	Speed/cost substitution, human inability to verify, automation bias	deployment choice, model propensity	Human evaluation too slow or expensive to keep up (Thakkar et al., 2025). Capability illegibility (Bowman et al., 2022). Humans defer when they would have wanted to be asked (Goddard et al., 2012). Evaluation becomes a delegated task (Ibrahim et al., 2025)
5. AI-AI bias	Explicit gatekeeping, implicit AI-AI bias (AI prefers AI more than humans do)	deployment choice, model propensity	Stated refusal to interoperate or preference for outputs labelled as AI (Laurito et al., 2025). Collusion in the wild against humans (Motwani et al., 2024). Differential resource/permission provision. Preference for AI-stylistic outputs (Panickssery et al., 2024). Ease-of-interaction routing that silently favours AI counterparties

anchoring-prone); the no-AI-reviewer stratum on its own is the cleanest single-difference read of human preference for AI-authored code. Within-repo twin-matching of AI- and human-authored PRs ($n \approx 4{,}000$ pairs available in our data) supports a difference-in-differences across reviewer type that purges shared code-quality confounders. We treat these as planned analyses; the present paper reports the descriptive within-PR comparison. (v) *Merger attribution.* We measure reviews, not merges. No merges in our universe are attributed to AI-bots, but AI can still act *through* a human merger when an AI approval gates the decision. (vi) *Accelerators, not outcomes. Accelerators* are observable, measurable patterns rather than disempowerment itself. They may also point in the opposite direction (e.g. switching costs can fall as well as rise).

4. Results

RQ1 — AI-agent participation is growing. Figure 1 shows monthly AI-agent participation rising from 7.5% of PRs in April 2025 to 18.1% in March 2026 (2.4×). The share of PRs containing an AI-AI comment chain of length ≥5 grew from 0.2% to 3.2%. The p95 longest-chain length doubled from 1 to 4 events. The trend is monotonic across twelve months. A single high-activity slice of GitHub cannot establish that AI-agent activity is accelerating ecosystem-wide, but within this slice the participation accelerator is clearly measurable and moving.

RQ2 (accelerator 4) — AI evaluation authority is growing. The share of PRs with an AI-agent approval/rejection decision rose from 0.01% to 0.01% over the year. The stricter "*only* AI decision, no human review" share rose from 0.00% to 0.01%. Among *merged* PRs, the share carrying an AI approval with no human approval rose from 0.00% to 0.00%, and by month's end exceeds the share of merged PRs carrying both an AI and a human approval (0.00% in March 2026). Both signals are lower bounds, because our allowlist may miss less-common AI bots. Combined with the AI-AI comment-chain growth, this is illustrative evidence that AI evaluation authority ($d\alpha_t/dt$) is measurable and increasing.

RQ3 (accelerator 5) — No strong evidence for AI-AI bias. A tighter within-PR test holds the PR (and implicitly, code quality) fixed. AI code-review tools in our sample overwhelmingly issue COMMENTED reviews rather than APPROVED ones: of 69,198 AI-bot review events, only 139 (≈0.2%) are approvals, and these come almost entirely from Cubic AI (n=85) and CodeRabbit (n=19). Copilot PR Reviewer, Gemini Code Assist, and Greptile post critiques but almost never approve. Using "AI reviewer" strictly to mean "AI bot that issued an APPROVED review", we ask whether a human reviewer and an AI approver, looking at the same PR, agree more readily when the PR is AI-authored. Of the 134 PRs where an AI bot approved, 0 (0.0%) were AI-authored. A human co-approved – of those, versus 66.4% of the 134 human-authored AI-approved PRs. The point-estimate gap (– pp) is directionally consistent with a small

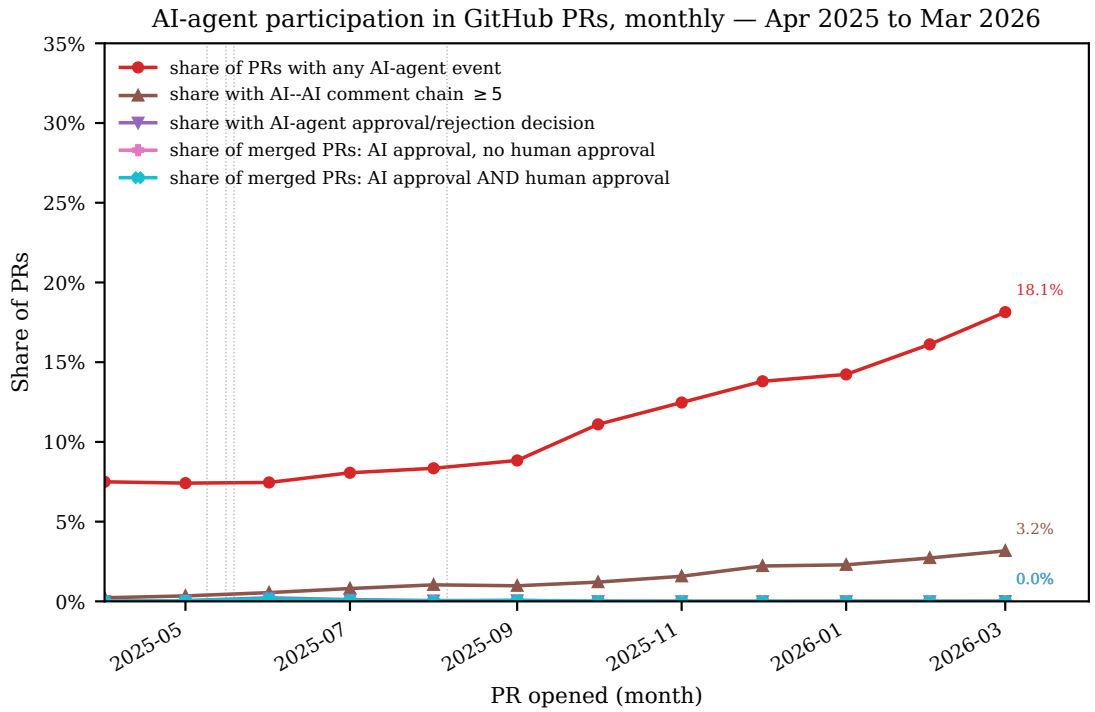

Figure 1. Monthly AI-agent participation in GitHub PRs across 2,000 popular repositories (Apr 2025–Mar 2026). Five series: share of PRs with any AI-agent event (RQ 1), share with an AI–AI comment chain of ≥ 5 contiguous events (RQ 1), share with an AI-agent approval or rejection decision (RQ 2), share of merged PRs with an AI approval and no human approval (RQ 2), and share of merged PRs with both an AI and a human approval (RQ 2). Denominators vary (all PRs or merged-only). First- and last-month values are annotated on every line. Vertical dotted lines mark approximate public-availability dates of several coding and code-review agents (reader context, not a causal claim).

AI-AI bias, but the two-proportion test is not significant (–) and the 95% CIs overlap substantially. **With a key cell of only 0 PRs, we treat this as an illustration of the test, not a finding**: we demonstrate that a within-PR AI-AI-bias measurement is feasible on public GitHub data. A larger approving-agent population is required before the direction can be read as a substantive claim.

5. Call to action

Treat accelerator tracking as a first-order empirical task, not a by-product of capability evaluation. Platforms (GitHub, Stack Overflow, npm, package registries) should publish monthly AI-agent activity reports. Model providers should include cross-family self-preference in post-training evaluations. Sectoral regulators should require the same for regulated agent markets. AI-mediated peer review (Thakkar et al., 2025), social sycophancy (Cheng et al., 2025), and anthropomorphic overreliance (Ibrahim & Cheng, 2025) all indicate that AI can shape human judgments without overt preference—our pipeline lets such effects, if they emerge at scale, be *tracked* rather than discovered retrospectively.

Disclosure of AI usage. AI assistants provided assistance with literature review, data analysis, and editing. The research idea, theoretical framework, and empirical approach are fully human-authored. Overall, the paper is primarily human-authored.

References

Agarwal, S., He, H., and Vasilescu, B. AI IDEs or autonomous agents? measuring the impact of coding agents on software development, 2026. URL https://arxiv.org/abs/2601.13597.

Appel, R., McCrory, P., Tamkin, A., McCain, M., Neylon, T., and Stern, M. Anthropic economic index report: Uneven geographic and enterprise AI adoption, 2025. URL https://arxiv.org/abs/2511.15080.

Bai, Y., Kadavath, S., Kundu, S., Askell, A., Kernion, J., Jones, A., Chen, A., Goldie, A., Mirhoseini, A., and

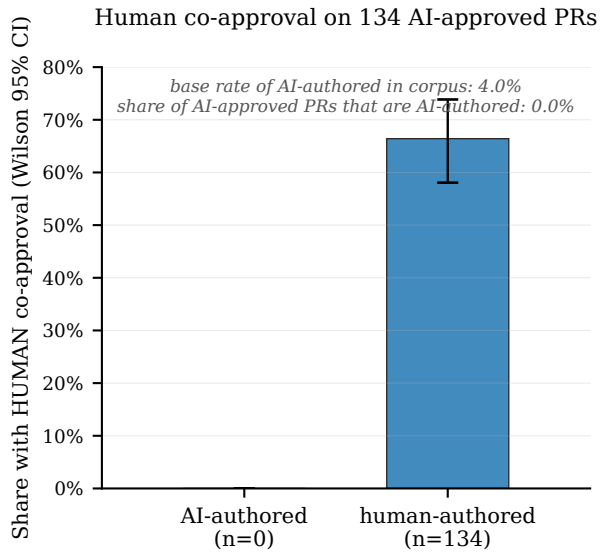

Figure 2. Human co-approval rate among the 134 PRs where an AI bot issued an APPROVED review. Red: AI-authored (n=0). Blue: human-authored (n=134). Point estimates differ by – percentage points (directionally consistent with AI reviewers approving AI code humans disagree with more often), but the two-proportion test is not significant (–) and the 95% CIs overlap — no statistically significant replication of the Laurito et al. (2025) single-turn AI-AI bias in this narrow slice. Error bars: Wilson 95% CIs. Not adjusted for code quality.

McKinnon, C. Constitutional AI: Harmlessness from AI feedback, 2022. URL https://arxiv.org/abs/2212.08073.

Borges, H. and Valente, M. T. What's in a GitHub star? Understanding repository starring practices in a social coding platform. *Journal of Systems and Software*, 146: 112–129, 2018. doi: 10.1016/j.jss.2018.09.016.

Bowman, S. R., Hyun, J., Perez, E., Chen, E., Pettit, C., Heiner, S., Lukošiūtė, K., Askell, A., Jones, A., and Chen, A. Measuring progress on scalable oversight for large language models, 2022. URL https://arxiv.org/abs/2211.03540.

Cheng, M., Yu, S., Lee, C., Khadpe, P., Ibrahim, L., and Jurafsky, D. ELEPHANT: Measuring and understanding social sycophancy in LLMs, 2025. URL https://arxiv.org/abs/2505.13995.

Chowdhury, K., Banik, D., Ferdous, K. M., and Shamim, S. I. From industry claims to empirical reality: An empirical study of code review agents in pull requests, 2026. URL https://arxiv.org/abs/2604.03196.

Collingridge, D. *The Social Control of Technology*. St. Martin's Press, New York, 1980.

Doshi, A. R. and Hauser, O. P. Generative AI enhances individual creativity but reduces the collective diversity of novel content. *Science Advances*, 10(28):eadn5290, 2024. doi: 10.1126/sciadv.adn5290.

Eguiluz Castañeira, J., Brando, A., Laukyte, M., and Serra-Vidal, M. Position paper: If innovation in AI systematically violates fundamental rights, is it innovation at all?, 2025. URL https://arxiv.org/abs/2511.00027. NeurIPS 2025 Position Paper track.

Ehsani, R., Pathak, S., Rawal, S., Al Mujahid, A., Imran, M. M., and Chatterjee, P. Where do AI coding agents fail? an empirical study of failed agentic pull requests in GitHub, 2026. URL https://arxiv.org/abs/2601.15195.

Ghaleb, T. A. Fingerprinting AI coding agents on GitHub, 2026. URL https://arxiv.org/abs/2601.17406.

Goddard, K., Roudsari, A., and Wyatt, J. C. Automation bias: A systematic review of frequency, effect mediators, and mitigators. *Journal of the American Medical Informatics Association*, 19(1):121–127, 2012. doi: 10.1136/amiajnl-2011-000089.

Huang, H., Jaisri, P., Shimizu, S., Chen, L., Nakashima, S., and Rodríguez-Pérez, G. More code, less reuse: Investigating code quality and reviewer sentiment towards AI-generated pull requests, 2026. URL https://arxiv.org/abs/2601.21276.

Ibrahim, L. and Cheng, M. Thinking beyond the anthropomorphic paradigm benefits LLM research, 2025. URL https://arxiv.org/abs/2502.09192.

Ibrahim, L., Collins, K. M., Kim, S. S. Y., Reuel, A., Lamparth, M., Feng, K., Ahmad, L., Soni, P., El Kattan, A., Stein, M., Swaroop, S., Sucholutsky, I., Strait, A., Liao, Q. V., and Bhatt, U. Measuring and mitigating overreliance is necessary for building human-compatible AI, 2025. URL https://arxiv.org/abs/2509.08010.

Juzek, T. S. and Ward, Z. B. Why does ChatGPT "delve" so much? Exploring the sources of lexical overrepresentation in large language models. In *Proceedings of the 31st International Conference on Computational Linguistics (COLING)*, pp. 6397–6411, 2025. URL https://aclanthology.org/2025.coling-main.426/.

Kalliamvakou, E., Gousios, G., Blincoe, K., Singer, L., German, D. M., and Damian, D. The promises and perils of mining GitHub. In *Proceedings of the 11th Working Conference on Mining Software Repositories (MSR)*, pp. 92–101, 2014. doi: 10.1145/2597073.2597074.

Kulveit, J., Douglas, R., Ammann, N., Turan, D., Krueger, D., and Duvenaud, D. Gradual disempowerment: Systemic existential risks from incremental AI development, 2025. URL https://arxiv.org/abs/2501.16946.

Laurito, W., Davis, B., Grieztes, P., et al. AI–AI bias: Large language models favor communications generated by large language models. *Proceedings of the National Academy of Sciences*, 2025. doi: 10.1073/pnas.2415697122. arXiv:2407.12856.

Li, H., Zhang, H., and Hassan, A. E. The rise of AI teammates in software engineering (SE) 3.0: How autonomous coding agents are reshaping software engineering, 2025. URL https://arxiv.org/abs/2507.15003.

Li, H., Zhang, H., and Hassan, A. E. AIDev: Studying AI coding agents on GitHub, 2026. URL https://arxiv.org/abs/2602.09185.

LogicStar AI. Agents in the wild: Measuring AI coding agent activity across open source, 2025. URL https://insights.logicstar.ai/. Live tracker.

Motwani, S. R., Baranchuk, M., Strohmeier, M., Bolina, V., Torr, P. H. S., Hammond, L., and de Witt, C. S. Secret collusion among AI agents: Multi-agent deception via steganography. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024. URL https://arxiv.org/abs/2402.07510.

Munaiah, N., Kroh, S., Cabrey, C., and Nagappan, M. Curating GitHub for engineered software projects. *Empirical Software Engineering*, 22(6):3219–3253, 2017. doi: 10.1007/s10664-017-9512-6.

OpenSSF Securing Critical Projects Working Group. ossf/criticality_score: Gives criticality score for an open source project. GitHub project, version 2.0, 2024. URL https://github.com/ossf/criticality_score. Accessed snapshot: gs://ossf-criticality-score/2025.07.25/01059/all.csv.

Panickssery, A., Bowman, S. R., and Feng, S. LLM evaluators recognize and favor their own generations, 2024. URL https://arxiv.org/abs/2404.13076.

Peng, S., Kalliamvakou, E., Cihon, P., and Demirer, M. The impact of AI on developer productivity: Evidence from GitHub Copilot, 2023. URL https://arxiv.org/abs/2302.06590.

Pike, R. and Lewandowski, A. Finding critical open source projects. Google Open Source Blog, 2020. URL https://opensource.googleblog.com/2020/12/finding-critical-open-source-projects.html.

Shao, Y., Zope, H., Jiang, Y., Pei, J., Nguyen, D., Brynjolfsson, E., and Yang, D. Future of work with AI agents: Auditing automation and augmentation potential across the U.S. workforce, 2025. URL https://arxiv.org/abs/2506.06576.

Sharma, M., McCain, M., Douglas, R., and Duvenaud, D. Who's in charge? disempowerment patterns in real-world LLM usage, 2026. URL https://arxiv.org/abs/2601.19062.

Shumailov, I., Shumaylov, Z., Zhao, Y., Papernot, N., Anderson, R., and Gal, Y. AI models collapse when trained on recursively generated data. *Nature*, 631:755–759, 2024. doi: 10.1038/s41586-024-07566-y.

Staufer, L., Feng, K., Wei, K., Bailey, L., Duan, Y., Yang, M., Ozisik, A. P., Casper, S., and Kolt, N. The 2025 AI agent index: Documenting technical and safety features of deployed agentic AI systems, 2026. URL https://arxiv.org/abs/2602.17753.

Stein, M. How are AI agents used? Evidence from 177,000 MCP tools, 2026. URL https://arxiv.org/abs/2603.23802.

Thakkar, N., Yuksekgonul, M., Silberg, J., Garg, A., Peng, N., Sha, F., Yu, R., Vondrick, C., and Zou, J. Can LLM feedback enhance review quality? a randomized study of 20K reviews at ICLR 2025, 2025. URL https://arxiv.org/abs/2504.09737.

van der Weij, T., Hofstätter, F., Jaffe, O., Brown, S. F., and Ward, F. R. AI sandbagging: Language models can strategically underperform on evaluations, 2024. URL https://arxiv.org/abs/2406.07358.

Watanabe, M., Li, H., Kashiwa, Y., Reid, B., Iida, H., and Hassan, A. E. On the use of agentic coding: An empirical study of pull requests on GitHub, 2025. URL https://arxiv.org/abs/2509.14745.

Zhong, S., Noei, S., Zou, Y., and Adams, B. Human-AI synergy in agentic code review, 2026. URL https://arxiv.org/abs/2603.15911.

A. Example pull requests

A.1. AI-author × AI-reviewer cell (approved-review subset, $n = 11$)

The thin AI×AI cell in Table **??** consists of the following eleven PRs. Eight are concentrated in a single repository (`thedotmack/claude-mem`), which is why the cell's merge rate is sensitive to one maintainer's behaviour. Of the eleven authors, only three use a bot login (`devin-ai-integration`, `copilot-swe-agent`). The remaining eight are human logins flagged as AI-author at high confidence via `Co-authored-by` commit trailers (see Appendix C).

- firecrawl/firecrawl#2515 — merged
- firecrawl/firecrawl#2602 — merged (`devin-ai-integration`)
- firecrawl/firecrawl#3183 — merged
- thedotmack/claude-mem#6 — not merged
- thedotmack/claude-mem#11 — merged (`copilot-swe-agent`)
- thedotmack/claude-mem#16 — merged (`copilot-swe-agent`)
- thedotmack/claude-mem#25 — merged
- thedotmack/claude-mem#31 — merged
- thedotmack/claude-mem#1454 — not merged
- thedotmack/claude-mem#1501 — not merged
- thedotmack/claude-mem#1542 — not merged

A.2. Longest AI→AI interaction chains

The PRs below tie for the longest contiguous run of high-confidence AI-bot events observed in our window (chain length 13, out of a 42,823-PR sample). *Chain length* counts the longest consecutive sequence of events by any high-confidence AI actor within one PR's time-ordered event stream (commits, reviews, comments, timeline events). See Section 3. Note that a chain can span multiple AI actors (e.g., a Copilot commit followed by a CodeRabbit review), and can also consist of a single bot repeating the same action type.

Chain length $= 13$ **(tied).**

- TanStack/query#8963
- appwrite/appwrite#10898 — AI author, AI reviewer, merged
- coleam00/Archon#915
- nocodb/nocodb#11486 — merged
- usebruno/bruno#5680
- usestrix/strix#328

High AI-event density (even when chain ≤ 11**).** These PRs contain an unusually high absolute count of AI-bot events, often interrupted by a single human event that caps the contiguous chain:

- vitejs/vite#20468 — 26 AI events, chain $= 11$, merged
- RocketChat/Rocket.Chat#38623 — 25 AI events, chain $= 10$, merged

Repository concentration. Repositories with the longest observed chain in the sample (`max`) and the summed chain length across all of their sampled PRs (`sum`). `calcom/cal.diy` stands out for sustained AI-on-AI activity rather than a single outlier PR.

repository	max chain	$\sum$ chain	PRs sampled
appwrite/appwrite	13	228	90
TanStack/query	13	183	106
coleam00/Archon	13	152	87
usestrix/strix	13	148	102
usebruno/bruno	13	149	84
nocodb/nocodb	13	17	90
microsoft/VibeVoice	12	65	109
calcom/cal.diy	12	**264**	110
wshobson/agents	12	19	74
DayuanJiang/next-ai-draw-io	11	120	101

B. Robustness and threats to validity

This appendix expands the limitations listed in the main paper.

B.1 Selection into the repository universe. We restrict to 2,000 repositories passing the "young-popular OR old-active-popular" filter (Appendix D). The resulting sample has median stars 53,583 and minimum stars 18,755 — it is effectively the top most-active highly-popular repositories on GitHub, not a uniform sample. Effects we estimate apply to "high-activity open-source repositories". We do not generalise to GitHub as a whole. In particular, chain-length growth in mid-popularity repositories (100–10,000 stars) may be smaller or larger than what we report, and we do not have data to say which.

B.2 Detection false-negatives: "undercover" agents. Our event-level detection (§3, Appendix C) relies on four signals: bot-account logins, `Co-Authored-By` trailers, AI configuration files, and handle mentions. Commercial coding agents have been reported to include configuration options that suppress attribution in outward artefacts — e.g., an "undercover mode" reportedly present in at least one major coding agent's system prompt, revealed via a source-code leak in early 2026. PRs produced by such configurations appear as ordinary human activity to our detector. Consequently the measured AI-agent participation share in Figure 1 is a *lower bound*. This does not overturn the paper's main claim (participation is growing) but tightens its interpretation: the true level is above what we measure, by an unknown amount.

B.3 Thin AI-approval slice in the within-PR test. Among our 2,000 repositories and 487,446 PRs, only 134 carry an approving review from an AI bot, and only 0 of those are also AI-authored. This is not a sampling accident: fewer than ten bots in our allowlist ever emit an "APPROVED" review state, and only two (Cubic, CodeRabbit) do so routinely. The two-proportion z-test on the 24 vs. 137 cells has limited power. We report it as a null on explicit self-preference, with the understanding that larger-sample follow-ups are needed to distinguish a small positive effect from zero at the ± 5 pp level.

B.4 Merger attribution: authors and reviewers, not mergers. We initially considered "AI merger" as a third dimension (author $\times$ reviewer $\times$ merger). No merges in our universe are attributed to AI-bot accounts on our allowlist, so the merger dimension collapses to "human" on every observed PR. This is a substantive limitation: a human clicks the merge button on every merged PR in our window, but AI can still act *through* that human—most obviously when an AI review's "approved" state is what gates a maintainer's decision to merge. Our observational design cannot separate a maintainer who merged *because of* an AI approval from one who merged in spite of it. We also caution that our detector excludes merges performed under a maintainer's personal-access token by an AI app or merge-queue automation acting on their behalf. Such merges would appear as human in both the REST `merged_by` field and the GraphQL timeline actor, and therefore cannot be recovered from the public event log alone.

B.5 Event timestamp ordering and chain definition. AI→AI chains are defined on events sorted by timestamp within a PR. GitHub timestamps are accurate to seconds. Ties are broken by the API's default order (commit $<$ review $<$ review-comment $<$ issue-comment). Alternative tie-breaking does not change the head of the distribution meaningfully (mean and p_{95} shift by $<$0.1 events). The tail (max chain = 13) is insensitive.

B.6 Reproducibility. Every number in the paper is derived from one of `data/pr_events.parquet` (one row per actor/event) or `data/chains.parquet` (one row per PR, chain aggregates). These are produced by `scripts/03_classify_prs.py` and `05_compute_chains.py` respectively. The within-PR two-proportion test is produced by `scripts/06c_within_pr.py`. The upstream PR-event data (`data/prs/*.jsonl`) is collected

by `scripts/02_fetch_prs.py` against the 2,000 repositories in `data/repos.json`, which itself is produced by `scripts/01_build_repo_list.py` (Appendix D details this step). Given a valid GitHub token, the full pipeline reproduces the paper's numbers up to GitHub's own indexing non-determinism.

C. Bot and AI-tool allowlists

Tables 2 and 3 list the GitHub bot accounts we classify as AI agents versus maintenance/CI bots excluded from analysis. The allowlist is intentionally conservative. Extending it is low-risk because adding a bot only moves PRs from `human`/`non_ai_bot` to `AI`, making the AI×AI cell larger.

Table 2. AI bot allowlist (as of April 2026).

Family	Account handles
Claude Code (Anthropic)	`claude[bot]`, `anthropic-ai[bot]`, `claude-bot`
GitHub Copilot	`copilot[bot]`, `github-copilot[bot]`, `copilot-swe-agent[bot]`
Devin (Cognition)	`devin-ai-integration[bot]`
Cursor Background Agent	`cursor-agent`, `cursor[bot]`, `cursoragent[bot]`
Google Jules	`jules-ai[bot]`, `jules[bot]`
Codegen.sh	`codegen-sh[bot]`
OpenHands	`openhands-agent[bot]`

Table 3. Non-AI bot exclusions (treated neither as human nor as AI).

`dependabot[bot]`, `renovate[bot]`, `snyk[bot]`, `greenkeeper[bot]`, `allcontributors[bot]`, `pre-commit-ci[bot]`, `stale[bot]`, `codecov[bot]`, `github-actions[bot]`, `mergify[bot]`, `semantic-release-bot`, `release-please[bot]`, `imgbot[bot]`, `deepsource-autofix[bot]`

Co-author trailer patterns (case-insensitive): `Co-Authored-By: Claude`, `Co-Authored-By: Copilot`, `Co-Authored-By: ChatGPT`, `...: Devin`, `...: Aider`, `...: Cursor`, `...: Cline/Claude-dev`, `...: Roo Code`, `...: Augment`, `...: Continue`, `...: Gemini/Google AI`, `...: Windsurf`, `...: OpenHands`, `...: Jules`.

D. Repository-selection pipeline (replication)

Overview. The repository universe is selected via the OpenSSF *criticality_score* (OpenSSF Securing Critical Projects Working Group, 2024; Pike & Lewandowski, 2020) rather than by raw GitHub stars. The score is a published, replicable ranking of GitHub projects by their structural importance to the open-source ecosystem; using it directly addresses the well-known critique that star-based selection over-weights short-lived viral projects relative to load-bearing infrastructure (Borges & Valente, 2018; Kalliamvakou et al., 2014; Munaiah et al., 2017). The selection is implemented in `scripts/01a_download_criticality.py` (downloads a pinned snapshot of the published score CSV) and `scripts/01_build_repo_list.py` (filters and enriches). The legacy v1 implementation, which used a star-bucket sweep, is preserved at `scripts/old_01_build_repo_list.py`; the resulting v1 universe is preserved at `data/old_phase1/repos.json`.

The OpenSSF criticality score. For each project, the score is a weighted arithmetic mean over normalised signals (OpenSSF Securing Critical Projects Working Group, 2024). The signals used by the default `all.csv` configuration are:

- `created_since`: months since first commit;
- `updated_since`: months since last commit (negatively weighted);
- `contributor_count`: distinct authors over the project's history;
- `org_count`: distinct organisations among contributors;

- `commit_frequency`: average commits per week (last year);
- `recent_release_count`: GitHub releases in the last year;
- `updated_issues_count` and `closed_issues_count`: issues touched in the last 90 days;
- `issue_comment_frequency`: comments per issue;
- `github_mention_count`: cross-repo references (in commit messages and other repos' code).

The score lies in $(0, 1]$; higher is more critical. The OpenSSF publishes a global score CSV to a public Google Cloud Storage bucket (`gs://ossf-criticality-score/`) with a new snapshot every two weeks. We pin a single snapshot (Step 1 below) so that re-running the pipeline reproduces the same universe.

Step 1 — Pin the criticality snapshot. We use the 2025.07.25 snapshot (`all.csv` variant, no deps.dev dependent counts), which contains 13,000 scored projects. The download script verifies the file by SHA-256 and writes a provenance record to `data/criticality/ossf-criticality-2025.07.25-all.csv.provenance.json` recording the source URL, byte size, hash, row count, and download timestamp. Concretely:

```
python scripts/01a_download_criticality.py \
    --snapshot-prefix 2025.07.25/010355 \
    --variant all.csv
```

Step 2 — Filter & sort by score. From the 13,000 rows we keep those with a numeric `default_score` and a `repo.url` starting with `https://github.com/`. After filtering: 13,000 rows. We sort descending by `default_score`, breaking ties by `repo.url` alphabetically for stability across reruns.

Step 3 — Candidate cap (over-sample). We take the top 13,000 candidates by score. The over-sample (vs. the final 2,000 target) absorbs drop-out in the next two filtering steps: repos that no longer exist on GitHub, were renamed to a private slug, or that are forks/archived/inactive.

Step 4 — GitHub enrichment. For each candidate we issue `GET /repos/{owner}/{name}` (REST API, async, 8-way bounded concurrency). The result populates the standard repo-metadata fields (stars, forks, `pushed_at`, license, default branch, etc.). HTTP 404 (deleted, renamed, or now-private) drops the row. After enrichment we reject:

- `fork == true` (forks contribute no independent PR activity);
- `archived == true`;
- `disabled == true`;
- `pushed_at` strictly before `2025-04-01` (no chance of in-window PRs).

Step 5 — Final cap. Of the survivors, we keep the top 2,000 by criticality score and write `data/repos.json`. Each row carries the GitHub-API fields plus the score, the within-snapshot rank, the snapshot date, and the raw OpenSSF signal columns (so any downstream re-weighting is possible without re-fetching the CSV).

Why 2,000 (not 10,000)? The v2 sample is set to the top 2,000 repositories by criticality score. The intermediate top-10,000 enrichment is preserved at `data/repos.top10k.json` for sensitivity analyses and for future scaling. The cost driver is Phase 2: collecting all PRs in the analysis window for 10,000 repos under the GitHub GraphQL rate limit takes multiple days, whereas 2,000 repos finishes within $\sim$24 h. Top-by-criticality is also the regime in which the score is most internally consistent (the long tail flattens out below 0.5875). Extending to a wider universe is one parameter change away (`data/repos.json` can be regenerated from `data/repos.top10k.json` by sorting on `criticality_rank` and slicing to the desired N).

Sample characteristics.

- 2,000 repositories selected from 13,000 scored candidates → 13,000 top-by-score → 12,977 successful GitHub enrichments → 11,307 after activity filter → 2,000 after cap.
- Criticality-score range: max 0.8487, min 0.5875.
- Stars: min 25, median 5,256, max 443,674.
- Top languages: TypeScript 345, JavaScript 320, Python 233, Java 171, C 155, Go 145, C++ 144, PHP 101, Rust 82, Ruby 49.
- GitHub API requests consumed during enrichment: 13,000.

PR volume in the v2 sample. Phase 2 (`scripts/02_fetch_prs.py`) collects every PR opened, updated, merged, or closed within the analysis window for each repository in `data/repos.json`, capped at 998 PRs per repo. The script records the true in-window count (`data/prs/_repo_pr_counts.json`) before any subsampling, so that the sample's PR-volume distribution is itself a reportable characteristic of the universe.

Off-GitHub workflows. Of the 2,000 repositories in the criticality-defined universe, 781 (39.0%) have zero in-window PRs. Inspection reveals these are predominantly Linux-kernel mirrors and Foundation-hosted projects whose canonical development happens off GitHub (LKML, git.apache.org, git.eclipse.org, etc.). The top-ranked example is `torvalds/linux` itself — ranked #2 by criticality with 230,982 GitHub stars but zero PRs, because Linux uses mailing-list patch submission. We retain such repositories in `data/repos.json` to faithfully report the criticality-derived universe, but they contribute no rows to downstream PR-event analyses. The *PR-active subsample* of 1,219 repositories is the analytic universe for Phases 3–7 of the pipeline; all PR-volume, chain-length, and merge-rate statistics in the body of this paper are computed over this subsample.

- Total in-window PRs across the v2 sample: 653,722 (mean 305.5, median 28 per repo; p90 1,199, max 1,495).
- Share of repositories with > 998 in-window PRs (i.e. those subsampled to the cap): 14.7% of the sample (314 of 2,000 repos). For these repositories the sampled rows are weighted toward the centre of the window by week-uniform subsampling rather than the most-recent months.
- Repositories at or below the cap have their full PR history within the window analysed; saturated repositories have 998 rows uniformly drawn over weeks.

Comparison with the v1 universe. The v1 selection (preserved at `data/old_phase1/repos.json`; see `paper/appendix/old_repo_selection.tex`) targeted 3,000 GitHub repositories that gained $\geq 1{,}000$ stars during 2025 and concentrated on hyper-popular consumer-visible projects. The v2 universe is broader: criticality emphasises long-lived, multi-org, heavily-cross-referenced projects, which includes many language and tooling foundations that have modest absolute star counts but are disproportionately depended-upon. The v2 sample therefore has a much wider star range and is closer to a defensible proxy for "the open-source ecosystem" than the v1 sample. The trade-off is that headline numbers computed on v1 do not transfer; Phases 2–7 must be re-run against `data/repos.json` for any results we report on the v2 sample.

Known selection biases. The criticality score is itself an opinionated metric: it favours older projects with multi-org contributor bases, recent commit activity, frequent releases, and inbound cross-repo references. Three biases inherited from this:

- Library-style projects score higher than application-style projects of comparable importance because they accumulate inbound mentions from their dependents' commit messages.
- Pure-binary or assets-only repositories with no commits-as-history (e.g. Wikipedia mirrors) are under-scored.
- The score is computed over GitHub-visible signals only; non-GitHub mirrors of important projects (e.g. many Apache and Eclipse Foundation projects whose canonical home is elsewhere) are absent or under-counted.

We do not attempt to correct for these. They are documented in §Limitations.

Reproducing `data/repos.json`. With a valid `GH_TOKEN` (or `gh auth login`) and the subproject's virtualenv activated:

```
python scripts/01a_download_criticality.py
python scripts/01_build_repo_list.py \
    --snapshot-date 2025.07.25 \
    --variant all.csv \
    --candidate-cap 13000 \
    --final-cap 10000 \
    --window-start 2025-04-01
```

Runtime: ∼30 s for the CSV download and parse, ∼30–120 min for the GitHub enrichment depending on rate-limit reset behaviour (each contiguous burst of ≈5,000 requests consumes the core hourly quota and is followed by a wait of up to one hour). 8-way concurrency keeps the per-burst wall clock under 5 minutes. The output is byte-for-byte reproducible up to GitHub's own indexing churn (a small number of borderline-archived repos may flip across reruns).

Replication artefacts. The pipeline writes:

- `data/criticality/ossf-criticality-2025.07.25-all.csv` — raw OSSF CSV.
- `data/criticality/ossf-criticality-2025.07.25-all.csv.provenance.json` — snapshot metadata, byte size, SHA-256, row count.
- `data/repos.json` — final 2,000-row repo list (downstream-compatible schema).
- `results/phase1_stats.json` — counts at every filtering step plus score and star distributions.
- `logs/01a_download_criticality.log` and `logs/01_build_repo_list.log` — per-script execution logs.

E. Supplementary figures

These supplementary figures back the two main-text figures (Fig. 1, Fig. 2) with more detail on distributional and cross-cell behaviour.

F. Data and methods (full description)

Repository universe. Public GitHub repositories plausibly gaining ≥1,000 stars in 2025 and actively pushed within our analysis window, capped at the top 2,000 by a recency-weighted activity score. Forks and archives are excluded. Non-goal: GitHub-wide representativeness. Full selection procedure in Appendix D.

PR universe. All PRs *created* between 2025-04-01 and 2026-03-31 in those repositories (487,446 PRs), fetched via GitHub GraphQL with `orderBy:CREATED_AT,DESC`. We record metadata, commits, reviews, review-thread comments, issue comments, and merge/close timeline events.

Event-level AI classification. Adapting Ghaleb (2026)'s heuristics to event granularity, we classify each actor–event pair as **AI-bot** (account on our 20-entry allowlist: `copilot-swe-agent[bot]`, `devin-ai-integration[bot]`, `cursor-agent`, `claude[bot]`, `coderabbitai`, `cubic-dev-ai`, `gemini-code-assist`, `copilot-pull-request-reviewer`, etc. – see Appendix C), **AI-assisted** (human account whose payload contains a `Co-Authored-By` trailer to an AI tool or a direct handle mention), or **human**. Confidence is *high* when a bot account or co-author trailer matches, *low* otherwise. Maintenance bots (Dependabot, Renovate, CI) are excluded. Primary analyses use high-confidence AI only.

Chain length (RQ1). Within a PR, order attributed events by timestamp. An *AI→AI chain* is a maximal contiguous subsequence of events with `actor_type=AI-bot` at high confidence. The per-PR metric is the longest such chain. We report the monthly share of PRs with chain ≥1, ≥2, ≥5.

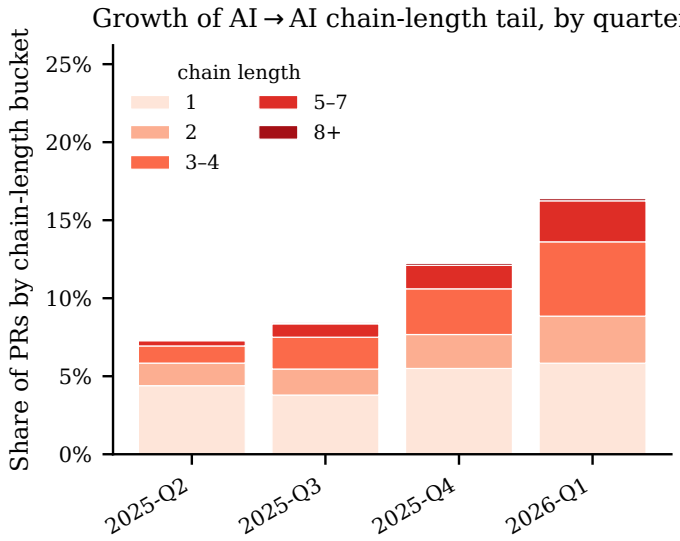

Figure 3. Distribution of longest AI→AI chain length per PR, by quarter. Boxes show the interquartile range with whiskers at 1.5×IQR. Fliers are omitted. Annotations report the quarterly mean and p_{95}. The tail grows monotonically: p_{95} triples from 2025-Q2 to 2026-Q1. The single-PR maximum (13) is stable, indicating growth driven by more *PRs* having long chains rather than any one PR having an unprecedentedly long chain.

Within-PR AI-AI bias test (RQ2). For each PR we assign `author_type` $\in$ {AI,human} (AI if the PR's author login is on our allowlist OR any commit bears an AI co-author trailer). Restricting to PRs where an AI bot issued an *approving* review, we test whether the human co-approval rate differs by author type with a two-sided two-proportion z-test (Wilson 95% CIs per cell).

Scope: authors and reviewers, not mergers. We measure AI participation through authoring and reviewing, not merging. No merges in our universe are attributed to AI-bot accounts on our allowlist, so "AI merger" is not an informative dimension in this window. We discuss the limitations of that framing—including the fact that AI can still act *through* a human merger—in §3.